

Meta 2 - Implementação do Sistema de Produções

Name: Flavia Salta

Student Number: 2025261469

Email: uc2025261469@student.uc.pt

Practical Class: PL6

Name: Diogo Saldanha

Student Number: 2022232761

Email: uc2022232761@student.uc.pt

Practical Class: PL6

The goal of this project is to develop a reactive agent capable of safely landing a spacecraft in the Lunar Lander environment. The agent must control the spacecraft using a set of observations and actions, ensuring a stable and safe landing within a designated area, controlling three engines: one main engine and two lateral thrusters.

Environment and Configuration

Throughout all experiments, we used:

- ENABLE_WIND: False → No wind force applied
- WIND_POWER: 15.0 → Wind intensity (inactive)
- TURBULENCE_POWER: 0.0 → Turbulence intensity (inactive)
- GRAVITY: -10.0 → Gravitational acceleration
- EPISODES: 1000 → Number of simulation episodes

The observation space provides 8 variables at each timestep:

1. horizontal position (x)
2. vertical position (y)
3. horizontal velocity (vx)
4. vertical velocity (vy)
5. orientation angle (theta)
6. angular velocity (v_theta)
7. boolean contact flag for the left landing leg (contact_left)
8. boolean contact flag for the right landing leg (contact_right)

Perceptions

Perceptions are computed functions over the raw observation vector that abstract the continuous state into conditions the production system can reason about.

is_touching_ground

Detects ground contact using the leg sensors.

Returns True if either landing leg is in contact with the ground (contact value > 0.5)

get_target_angle

Calculates a dynamic target angle based on the horizontal position (x) and horizontal velocity (v_x) to guide the ship toward the center of the landing pad (coordinates 0,0).

is_falling_too_fast

Returns True if the vertical descent speed is high. It monitors vertical velocity (v_y) relative to altitude (y) to ensure the landing meets the requirement of $v_y > -0.2$

is_unsafe_to_thrust

Prevents the main engine from firing if the tilt (θ) is too high to avoid accidental horizontal acceleration.

Actions

Actions are predefined output vector passed to the environment at each step.

- **action_fire_main:** [1.0, 0.0] → Full main engine thrust upward
- **action_rotate_left:** [0.0, -1.0] → Activate right thruster, rotate left
- **action_rotate_right:** [0.0, 1.0] → Activate left thruster, rotate right
- **action_do_nothing:** [0.0, 0.0] → All engines off

In practice, the reactive agent doesn't use the four functions in isolation. It directly computes continuous values for main engine and side engines, which allows finer-grained control. The actions above represent the extremes of the action space and serve as a conceptual design tool.

Production System

The reactive agents implement a priority ordered production system. Rules are evaluated top-down and the first matching condition determines the output. The system is structured into three logical layers: a safety kill switch, an altitude-based descent controller and a rotation/position correction system.

Kill Switch

IF is_touching_ground(observation) THEN return [0.0, 0.0]

As soon as any leg touches the ground, all engines are shut down immediately. This prevents the lander from reigniting engines after first contact, which would cause it to bounce or flip. This rule always takes priority over all others.

Altitude-Progressive Descent Control

The descent phase is divided into three altitude zones, each with its own braking threshold and engine power level. This approach ensures a gradual and controlled descent rather than a single fixed braking strategy.

- High ($y > 0.3$) $\rightarrow$ if $v_y < -0.4 \rightarrow 1.0$ (full thrust)
- Mid ($0.1 < y \leq 0.3$) $\rightarrow$ if $v_y < -0.2 \rightarrow 0.6$ (moderate thrust)
- Low ($y \leq 0.1$) $\rightarrow$ if $v_y < -0.05 \rightarrow 0.5$ (soft landing) Additionally, if the ship's tilt triggers the safety cutoff ($\text{abs}(\theta) > 0.5$), the main engine thrust is set to 0.0 to prevent it from launching itself sideways.

Rotation and Position Correction

After the descent rules are applied the lateral thruster value is computed based on the current angular velocity (v_θ) and the error between current angle (θ) and the target angle computed by `get_target_angle(x, vx)`.

- $v_\theta > 0.15 \rightarrow 1.0$ (Dampen counter-clockwise spin)
- $v_\theta < -0.15 \rightarrow -1.0$ (Dampen clockwise spin)
- $\theta < \text{target} - 0.05 \rightarrow -1.0$ (Correct angle: rotate left)
- $\theta > \text{target} + 0.05 \rightarrow 1.0$ (Correct angle: rotate right)
- Otherwise $\rightarrow 0.0$ (No rotation needed)

Implementation Notes

The agent is implemented in the `reactive_agent` function. It reads the 8 observation values, applies the production rules described above, and return a numpy array.

- Continuous action space: unlike a discrete agent, this implementation uses partial thrust values (0.5/0.6) for smoother control and gentler landings.
- Separation of concerns: perception functions are cleanly separated from the production rules, making the agent easier to debug and modify.
- Threshold tuning: values such as the tilt safety threshold (0.4 rad), descent velocity limits, and target angle aggressiveness (0.1 rad) were tuned empirically through iterative testing.
- Kill switch robustness: using > 0.5 instead $== 1$ for leg contact detection avoids false negatives due to floating-point representation.

Experiments and Results

A total of 100 episodes were run with `RENDER_MODE = humans` but to allow fast batch evaluation we used `RENDER_MODE = None` (headless mode). The seed was set to `None` so each episode uses a different random initial configuration, providing a representative evaluation across varied starting conditions.

Total episodes: 1000

Successful Landings: ~428

Success Rate: ~48.2%

Average Steps (successful): ~40,124

Wind Enables: No

The agent achieved a success rate of approximately 42.3% over 1000 episodes. This result is consistent with the stochastic nature of the environment. The lander spawns with a random initial position and orientation and some configurations are significantly more challenging than others.

Results varied slightly between runs (typically between 39% and 43%) due to this randomness, which is expected.

The average of approximately 28,846 steps per successful landing reflect that the agent takes a conservative approach: it carefully manages altitude and rotation before committing to a descent, which results in longer but more controlled landings.

If we set `ENABLE_WIND = True`, the environment adds a horizontal wind force that pushes the lander sideways randomly during flight.

The agent drops to ~8.6% success rate. To handle the wind we need a more aggressive horizontal correction in `get_target_angle` and a stronger angular damping.

Iterative Development

Several design iterations were performed before reaching the final version. An initial version used a single fixed braking threshold regardless of altitude, which frequently caused the lander to hover too high before cutting thrust and then falling uncontrolled. The introduction of the three-zone altitude system significantly improved success rates. Angular velocity damping was added after observing that the lander oscillated uncontrollably when only angle correction was used. The tilt safety cutoff for the main engine ($|\theta| > 0.5$) was introduced to prevent a failure mode where the lander would fire its main engine while severely tilted, launching itself sideways. The target angle aggressiveness was also increased from 0.1 to 0.15 radians to improve horizontal drift correction, and the deadband for rotation correction was reduced from 0.05 to 0.03 radians for finer orientation control. Finally, lowering the high-altitude braking trigger from $v_y < -0.4$ to $v_y < -0.35$ provided the margin needed to push the success rate above 40%.

Conclusions

The final agent achieves a success rate around 40% across 1000 episodes without wind. The main limitations of the reactive approach are that: the agent can't plan ahead, anticipate future states or adapt to the specific characteristics of each episode.

It demonstrates the strengths and weaknesses of reactive systems: they are simple to implement, cheap and robust to real-time constraints, but are fundamentally limited by the complexity of the behaviors they can express through hand crafted rules alone.